

การประกอบ SQL Query ในระบบ Migration Validator

เอกสารสรุปวิธีที่ระบบตรวจสอบข้อมูล (MySQL 4 → 8 Migration Validator) สร้างและรัน SQL query จริงในแต่ละโหมด

อ้างอิงโค้ดจริงใน `validate_all_tables.py` และ `web_server.py` (branch `multi_compare`)

1. ภาพรวม: ใครทำหน้าที่อะไร

จุดสำคัญที่สุดที่ต้องเข้าใจก่อน: SQL ที่ยังไปยัง 2 ฐานข้อมูล (source = MySQL 4.0, target = MySQL 4 หรือ 8) มีหน้าที่แค่ "ดึงแถวออกมาตามลำดับ" เท่านั้น ไม่มี query ไหนสั่งให้ฐานข้อมูลเทียบข้อมูลกันเอง (ไม่มี cross-database JOIN เพราะเป็นคนละ connection/เครื่องกันเสมอ) — logic การเทียบข้อมูลทั้งหมดอยู่ในโค้ด Python ฝั่ง client

- SQL ฝั่ง DB: ดึงแถวตามลำดับ primary key (keyset pagination) — ให้ทั้งสองฝั่งได้แถวมาเรียงลำดับเดียวกัน
- Python ฝั่ง client: เดินสอง generator พร้อมกัน (two-pointer merge), จับคู่แถวด้วย PK, เทียบทีละฟิลด์, normalize type/date ก่อนเทียบ, จัดผลเป็น MATCH / MISMATCH / MISSING / EXTRA

2. Query แยกตามโหมดตรวจสอบ

2.1 Row Count Check

นับจำนวนแถวสองฝั่งเทียบกัน โดยใช้ query เดียวกันทั้งสองฝั่ง:

```
SELECT COUNT(*) FROM `{table}`{where}
```

ไฟล์: `validate_all_tables.py`:427, 435 — {where} ใส่เฉพาะเมื่อมี time-window filter

2.2 Schema Check

เทียบชนิดและขนาดคอลัมน์ เนื่องจาก MySQL 4 ไม่มี `information_schema` เต็มรูปแบบ จึงมีการแยกวิธีดึงข้อมูล:

```
-- Source (MySQL 4.0) และ Target MySQL 4:
SHOW COLUMNS FROM `{table}`

-- Target MySQL 8:
SELECT column_name, column_type, is_nullable, column_key
FROM information_schema.columns
WHERE table_schema = %s AND table_name = %s
ORDER BY ordinal_position
```

ไฟล์: `validate_all_tables.py`:567 (source), 575-578 (target v4), 580-587 (target v8) — ก่อนเทียบจะ normalize type ก่อน (ตัด precision จาก `double(x,y)/float(x,y)`, ตัด display width จาก `int(11)` → `int`)

2.3 Index Check

```
SHOW INDEX FROM `{table}`
```

ไฟล์: validate_all_tables.py:672 (source), 683 (target) — รูปแบบผลลัพธ์เหมือนกันทั้งสองเวอร์ชัน กลุ่มตาม index name แล้วเรียงตาม seq_in_index

2.4 Duplicate PK Check

ไม่มี query เลย — อาศัย PRIMARY KEY constraint ของฐานข้อมูลเองบังคับ unique อยู่แล้ว ตอบ PASS ทันที (validate_all_tables.py:722-729) หลีกเลี่ยง COUNT(DISTINCT) ที่ช้าและเสี่ยง timeout กับตารางใหญ่

2.5 Full Sync Check (เทียบทีละแถวที่ละฟิลด์ 100%)

ใช้ keyset pagination (ORDER BY pk + LIMIT ไล่หน้า) แทน OFFSET เพื่อให้ DB เดินตาม index ได้ ไม่ต้อง sort ทั้งก่อนเข้าทุกหน้า ดูรายละเอียดเต็มในหัวข้อ 3

2.6 Quick Scan

ไม่ใช่ query แยกใหม่ — แค่ "ข้าม Full Sync" รันเฉพาะ Row Count + Schema + Index (ข้อ 2.1-2.3) เมื่อตั้ง flag Skip Full Sync (validate_all_tables.py:1195-1196, 1269-1275)

2.7 Random Field-by-Field Comparison (สุ่มตัวอย่าง)

Route: POST /api/random-compare (web_server.py:818-1100)

PK ตัวเลขเดียว (range mode):

```
SELECT MIN(`{range_pk}`), MAX(`{range_pk}`) FROM `{table}`{window}

SELECT {col_list} FROM `{table}` WHERE `{range_pk}` >= {start_pk}{window_and}
ORDER BY `{range_pk}` LIMIT {sample_size}
```

PK ผสมหลายคอลัมน์ (anchor mode):

```
SELECT COUNT(*) FROM `{table}`{window}
SELECT {pk_list} FROM `{table}`{window} ORDER BY {order_by} LIMIT 1 OFFSET {offset}
SELECT {col_list} FROM `{table}` WHERE {pk_after_anchor}{window_and}
ORDER BY {order_by} LIMIT {sample_size}
```

seed เดียวกัน → target ดึง slice เดียวกันเป๊ะกับ source (web_server.py:934-985)

2.8 SQL Runner (query ที่ user พิมพ์เอง)

Route: POST /api/custom-query (web_server.py:1323-1395) — whitelist เฉพาะ SELECT / SHOW / DESCRIBE / DESC / EXPLAIN เติม LIMIT อัตโนมัติถ้า SELECT

ไม่มี LIMIT (สูงสุด 2000 แถว) ไม่มีการแก้ syntax อื่นใด ไม่ block GROUP BY

2.9 SQL Compare (เทียบตามเงื่อนไขที่ user กำหนดเอง)

Route: POST /api/sql-compare, worker หลักที่ web_server.py:2005-2293

- ตัด ORDER BY/LIMIT เดิมออกจาก query ที่ user เขียน เก็บ LIMIT เดิมไว้ใช้ทีหลัง (2021-2025)
- รัน {strip_sql} LIMIT 1 ทั้งสองฝั่งหาชื่อคอลัมน์ แล้วหา intersection ของคอลัมน์ที่มีทั้งสองฝั่ง (2050-2069)
- หา PK จริงจาก schema ปลายทาง ตรวจสอบ string type เพื่อห่อ BINARY
- ประกอบ query สุดท้าย: {strip_sql} ORDER BY {pk_order} {limit_clause} (2111-2122)

ข้อควรระวัง: ถ้า query ต้นฉบับมี GROUP BY การเติม ORDER BY pk ต่อท้ายจะพังหรือได้ผลไม่ตรงเจตนา เพราะ SQL Compare ออกแบบมาสำหรับเทียบแถวดิบทีละ

แถวด้วย PK ไม่รองรับ query แบบ aggregate

3. Full Sync แบบละเอียด — Dynamic Query Template

Template เต็มที่ใช้ประกอบ query ทุกตาราง ไม่มีจุดไหน hardcode ชื่อตาราง/คอลัมน์ — ทุกตัวแปรมาจาก schema + config ที่อ่าน runtime ทั้งหมด

```
SELECT {col_list}
FROM `{table}`
WHERE {ts_col} >= '{WINDOW_START}'      -- ใส่เฉพาะถ้ามี ts_col + window
    AND {ts_col} < '{WINDOW_END}'        -- ใส่เฉพาะถ้าตั้งวันสิ้นสุดด้วย
    AND {keyset_boundary}                 -- ใส่เฉพาะหน้า 2 เป็นต้นไป
ORDER BY {pk_cols_ordered}
LIMIT {KEYSET_CHUNK}
```

จุดประกอบจริงรวมทุกตัวแปร: validate_all_tables.py:832-864 ฟังก์ชัน _keyset_stream

3.1 ตัวแปรแต่ละตัวมาจากไหน

ตัวแปร	ที่มาของข้อมูล (Runtime)	บรรทัด
col_list	all_cols ลบ ignore_fields (แต่เก็บ PK ไว้เสมอ)	1243-1245
table	ชื่อตารางที่กำลังตรวจสอบใน Loop	1236
ts_col	ts_field_config.json หรือได้จาก Date Type PK	224
WINDOW_START/END	คำนวณจาก tuning.data_window_years	247-256
keyset_boundary	pk_cols + ค่า PK แลสุดท้ายของหน้าก่อน	751-775
pk_cols_ordered	PK ตามลำดับ Seq_in_index (ห่อ BINARY ถ้าเป็น String)	842-845
KEYSET_CHUNK	tuning.keyset_chunk_size (Default 50,000)	738

3.2 Keyset boundary สร้างยังไง

จำลอง row-constructor (pk1,pk2,pk3) > (last1,last2,last3) ที่ MySQL 4 ไม่รองรับ ด้วยการทางเป็น OR ของ "เท่ากันหมดจนถึงตัวก่อนหน้า + ตัวนี้มากกว่า":

สำหรับแต่ละตำแหน่ง i ใน `pk_cols` (0 ถึง $n-1$):

เงื่อนไข _{i} = (`pk[0]=last[0]` AND ... AND `pk[i-1]=last[i-1]`) AND `pk[i] > last[i]`

ผลลัพธ์ = เงื่อนไข₀ OR เงื่อนไข₁ OR ... OR เงื่อนไข_{ $n-1$ }

ตัวอย่าง PK 3 คอลัมน์ (`symbol`, `date`, `seq`) แถวสุดท้ายของหน้าก่อน = ('PTT','2024-03-15',2):

```
( `symbol` > 'PTT' )  
OR ( `symbol` = 'PTT' AND `date` > '2024-03-15' )  
OR ( `symbol` = 'PTT' AND `date` = '2024-03-15' AND `seq` > 2 )
```

เทียบเท่า `lexicographic comparison (symbol, date, seq) > ('PTT','2024-03-15',2)` — ตัวแรกไม่เท่ากันตัดสินเลย เท่ากันค่อยดูตัวถัดไป ไล่:

`validate_all_tables.py:751-775` ฟังก์ชัน `_keyset_cond`

BINARY ที่แทรกบางจุด: ใส่เฉพาะคอลัมน์ PK ที่เป็น string type บังคับเทียบแบบ byte-value เพราะ MySQL 4 (latin1) กับ MySQL 8 (utf8mb4) เรียง string คนละแบบ —

ไม่ใช่จะทำให้ two-pointer merge จับคู่แถวผิด

3.3 ตัวอย่าง query จริงเต็มรูปแบบ

ตาราง `daily_price` (`ts_col` = timestamp ตาม `ts_field_config.json`), PK = (`symboltypeid`, `share`), `tuning.data_window_years` = 2 ปี 2569:

หน้าแรก:

```
SELECT `share`, `closeprice`, `openprice`, `high`, `low`, `avgprice`,  
       `volumetrade`, `valuetrade`, `timestamp`, `buyvolumetrade`,  
       `sellvolumetrade`, `buyvaluetrade`, `sellvaluetrade`, `symboltypeid`  
FROM `daily_price`  
WHERE `timestamp` >= '2025-01-01'  
ORDER BY `symboltypeid`, BINARY `share`  
LIMIT 50000
```

หน้าถัดไป (แถวสุดท้ายของหน้าก่อน: `symboltypeid=1`, `share='PTT'`):

```
SELECT `share`, `closeprice`, ... , `symboltypeid`  
FROM `daily_price`  
WHERE `timestamp` >= '2025-01-01'  
      AND ( (`symboltypeid` > 1)  
            OR (`symboltypeid` = 1 AND BINARY `share` > BINARY 'PTT') )  
ORDER BY `symboltypeid`, BINARY `share`  
LIMIT 50000
```

หมายเหตุ: ลำดับ PK (`symboltypeid` ก่อน `share`) อ้างอิงตามที่ทีมยืนยันมา ควรเช็ค `SHOW INDEX` จริงอีกครั้งก่อนอ้างอิงถาวรเพราะไม่ได้ต่อ DB จริงตอนเขียนเอกสารนี้

4. Full Sync แบบละเอียด — ขั้นตอนรัน + เทียบ

ยิง query เดียวกันปะทะทั้งสองฝั่ง แล้วเทียบฝั่ง Python (ไม่มี query โทนเทียบข้าม DB ให้)

- Source (MySQL 4.0): ยิงผ่าน mysql40.py — ใช้ query() แบบเรียกซ้ำเป็นรอบทีละหน้า (ไม่ใช่ stream ต่อเนื่อง) ยิง → รอผลเต็มหน้า → return list → ยิงหน้าถัดไป
- Target (MySQL 8 หรือ 4): ยิงผ่าน MySQLdb cursor ปกติ (หรือ mysql40.py ถ้า target เป็น MySQL 4 ด้วย)
- Two-pointer merge: เดิน generator สองฝั่งพร้อมกันตามลำดับ PK ที่เรียงมาแล้ว จับคู่แถว PK ตรงกัน
- เทียบทีละฟิลด์ด้วย tolerance ตาม tuning.decimal_tol / decimal_round
- บันทึกผล MISMATCH / MISSING / EXTRA ลง CSV + NDJSON

4.1 mysql40.py — วิธีดึงข้อมูลระดับ protocol

mysql40.py เขียน MySQL wire protocol เองทั้งหมด เพราะ driver ทั่วไป (mysqlclient/pymysql) ต่อ MySQL 4.0 ไม่ได้ มี 2 ฟังก์ชันหลัก ใช้คนละโหมด:

ฟังก์ชัน	ใช้ในโหมดไหน	ลักษณะการทำงาน
query()	Full Sync (เรียกซ้ำทีละหน้า)	ส่ง query ครั้งเดียว รอผลครบทุกแถวของ LIMIT นั้นแล้ว return list เดียว — ถูกเรียกซ้ำเป็นรอบๆ จาก _keyset_stream
query_stream()	SQL Compare (query เดียวไม่แบ่งหน้า)	ส่ง query ครั้งเดียว อ่าน response แบบ yield ทีละแถวจาก socket ตรงๆ (generator) ไม่รอโหลดทั้งก้อนเข้า memory

ขั้นตอน protocol ระดับต่ำ (เหมือนกันทั้งสองฟังก์ชัน):

- ส่ง COM_QUERY packet (0x03 + sql) ไปตรงๆ ผ่าน raw socket
- อ่าน header packet — byte แรก = 0xFF แปลว่า error decode error message ทันที
- อ่านจำนวนคอลัมน์ (length-encoded int) วน _read_packet ทั่วทั้งคอลัมน์ (ข้าม column definition)
- อ่าน EOF packet (จบส่วน column defs)
- วนอ่านแถวข้อมูล — byte แรก 0xFE และ packet สั้น < 9 byte = EOF (จบผลลัพธ์) หยุด
- แต่ละ field: byte 0xFB = NULL, ไม่งั้น byte แรกบอกความยาว string (ถ้า ≥ 0xFC ต้องอ่านเพิ่ม 2-3 byte เป็นความยาวจริง) แล้ว decode ตาม charset (ปกติ tis620)

จุดสำคัญ: ทุกค่ากลับมาเป็น string เสมอ ไม่มี type casting ในขั้น protocol นี้ ตัวเลข/วันที่ต้อง cast เองใน Python ชั้นบน — นี่คือการมาของ normalize

type/date ที่ต้องทำก่อนเทียบทุกครั้ง

5. MySQL 4 vs MySQL 8 — quirks ที่วนเจือทุกโหมด

ประเด็นความต่าง	รายละเอียดทางเทคนิค
Schema Metadata	MySQL 4 ใช้ SHOW COLUMNS แทน information_schema
String Ordering	ต้องใช้ BINARY เพื่อให้การเรียงลำดับ byte ตรงกันทั้ง 2 ฝั่ง
Date/Time casting	MySQL 4 คำนวณค่าเป็น datetime เสมอ ต้อง normalize ก่อนเทียบ
Row Constructor	MySQL 4 ไม่รองรับ (a,b) > (x,y) ต้องกางเป็น OR/AND chain

6. แผนที่ไฟล์/ฟังก์ชัน สำหรับไปดูโค้ดต่อ

ส่วนไหน	ฟังก์ชัน / route	ไฟล์:บรรทัด
Row Count	check_row_count	validate_all_tables.py:420-435
Schema	check_schema	validate_all_tables.py:567-587
Index	check_indexes	validate_all_tables.py:672-683
Duplicate PK	check_duplicate_pk	validate_all_tables.py:722-729
Keyset boundary	_keyset_cond	validate_all_tables.py:751-775
ประกอบ query ต่อหน้า	_keyset_stream	validate_all_tables.py:832-864
Prefix partition	_prefix_keyset_stream	validate_all_tables.py:781-830
ควบคุม stream + merge	_stream_compare	validate_all_tables.py:867+
Random Sample	/api/random-compare	web_server.py:818-1100
SQL Runner	/api/custom-query	web_server.py:1323-1395
SQL Compare	/api/sql-compare	web_server.py:2005-2293

หมายเหตุ: เนื้อหานี้อ้างอิงจากการอ่านโค้ดจริงในโปรเจกต์ (branch multi_compare) ณ วันที่จัดทำเอกสาร ตัวอย่างลำดับ PK ในหัวข้อ 3.3 เป็นข้อมูลที่ทีมอื่นยื่นมา ยังไม่ได้ verify กับ SHOW INDEX จริง —
แนะนำให้เช็คอีกครั้งก่อนใช้อ้างอิงถาวร